

BÀI THỰC HÀNH MEDIATOR PATTERN

- ✓ Backend: ASP.NET Core Web API
 - ✓ Frontend: React (user thao tác giỏ hàng)
 - ✓ Có Mediator đúng chuẩn GoF
 - ✓ Giảm coupling giữa các component (Cart, Payment, Notification)
-

BÀI TOÁN

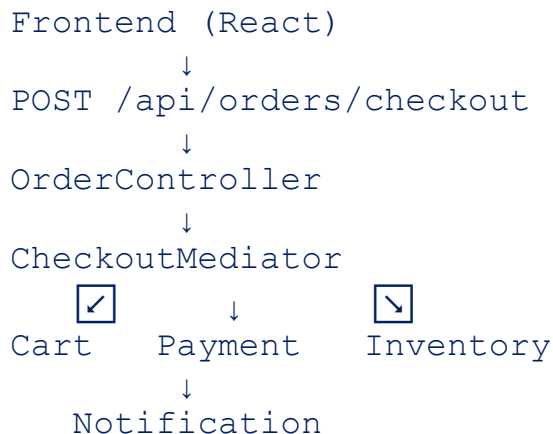
Hệ thống E-commerce khi checkout sẽ có nhiều thành phần tương tác:

- CartService
- PaymentService
- InventoryService
- NotificationService

👉 Nếu gọi trực tiếp → **coupling cao, khó maintain**

👉 Giải pháp: dùng **Mediator Pattern** để điều phối.

KIẾN TRÚC HỆ THỐNG (MEDIATOR)



👉 Các service **KHÔNG** gọi trực tiếp nhau

👉 TẤT CẢ đi qua Mediator

BACKEND – ASP.NET Core 8 Web API

Tạo project

```
dotnet new webapi -n MediatorDemo
cd MediatorDemo
```

CẤU TRÚC THƯ MỤC

```
MediatorDemo
├── Controllers
│   └── OrdersController.cs
├── Mediators
│   ├── ICheckoutMediator.cs
│   └── CheckoutMediator.cs
├── Services
│   ├── CartService.cs
│   ├── PaymentService.cs
│   ├── InventoryService.cs
│   └── NotificationService.cs
├── Models
│   ├── CheckoutRequest.cs
│   └── CheckoutResult.cs
└── Program.cs
```

1. REQUEST + RESPONSE MODEL

Models/CheckoutRequest.cs

```
namespace MediatorDemo.Models;

public class CheckoutRequest
{
    public decimal Amount { get; set; }
}
```

Models/CheckoutResult.cs

```
namespace MediatorDemo.Models;

public class CheckoutResult
{
}
```

```
        public bool Success { get; set; }
        public string Message { get; set; }
    }
```

❧ 2. CÁC SERVICE (KHÔNG BIẾT NHAU)

Services/CartService.cs

```
namespace MediatorDemo.Services;

public class CartService
{
    public bool ValidateCart()
    {
        return true;
    }
}
```

Services/InventoryService.cs

```
namespace MediatorDemo.Services;

public class InventoryService
{
    public bool ReserveItems()
    {
        return true;
    }
}
```

Services/PaymentService.cs

```
namespace MediatorDemo.Services;

public class PaymentService
{
    public async Task<bool> ProcessPayment(decimal
amount)
    {
        await Task.Delay(500);
        return true;
    }
}
```

Services/NotificationService.cs

```
namespace MediatorDemo.Services;

public class NotificationService
{
    public void Send(string message)
    {
        Console.WriteLine($"[Notification] {message}");
    }
}
```

✿ 3. MEDIATOR INTERFACE

Mediators/ICheckoutMediator.cs

```
using MediatorDemo.Models;

namespace MediatorDemo.Mediators;

public interface ICheckoutMediator
{
    Task<CheckoutResult> Checkout(decimal amount);
}
```

✿ 4. CONCRETE MEDIATOR (TRÁI TIM HỆ THỐNG)

Mediators/CheckoutMediator.cs

```
using MediatorDemo.Models;
using MediatorDemo.Services;

namespace MediatorDemo.Mediators;

public class CheckoutMediator : ICheckoutMediator
{
    private readonly CartService _cart;
    private readonly InventoryService _inventory;
    private readonly PaymentService _payment;
    private readonly NotificationService _notification;

    public CheckoutMediator(
        CartService cart,
        InventoryService inventory,
        PaymentService payment,

```

```

        NotificationService notification)
    {
        _cart = cart;
        _inventory = inventory;
        _payment = payment;
        _notification = notification;
    }

    public async Task<CheckoutResult> Checkout(decimal
amount)
    {
        // 1. Validate cart
        if (!_cart.ValidateCart())
        {
            return new CheckoutResult
            {
                Success = false,
                Message = "Cart invalid"
            };
        }

        // 2. Reserve inventory
        if (!_inventory.ReserveItems())
        {
            return new CheckoutResult
            {
                Success = false,
                Message = "Out of stock"
            };
        }

        // 3. Payment
        var paid = await _payment.ProcessPayment(amount);

        if (!paid)
        {
            return new CheckoutResult
            {
                Success = false,
                Message = "Payment failed"
            };
        }

        // 4. Notify
        _notification.Send("Order placed successfully");

        return new CheckoutResult

```

```
        {
            Success = true,
            Message = $"Order success - Paid {amount}"
        };
    }
}
```

❁ 5. CONTROLLER

Controllers/OrdersController.cs

```
using Microsoft.AspNetCore.Mvc;
using MediatorDemo.Models;
using MediatorDemo.Mediators;

namespace MediatorDemo.Controllers;

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly ICheckoutMediator _mediator;

    public OrdersController(ICheckoutMediator mediator)
    {
        _mediator = mediator;
    }

    [HttpPost("checkout")]
    public async Task<IActionResult>
    Checkout(CheckoutRequest request)
    {
        var result = await
        _mediator.Checkout(request.Amount);
        return Ok(result);
    }
}
```

❁ 6. ĐĂNG KÝ DI

Program.cs

```
using MediatorDemo.Mediators;
using MediatorDemo.Services;
```

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

// Services
builder.Services.AddTransient<CartService>();
builder.Services.AddTransient<InventoryService>();
builder.Services.AddTransient<PaymentService>();
builder.Services.AddTransient<NotificationService>();

// Mediator
builder.Services.AddTransient<ICheckoutMediator,
CheckoutMediator>();

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowReact",
        policy =>
        {
            policy.WithOrigins("http://localhost:3000")
                .AllowAnyHeader()
                .AllowAnyMethod();
        });
});

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseCors("AllowReact");
app.UseHttpsRedirection();
app.MapControllers();

app.Run();
```

TEST API

POST /api/orders/checkout

Body:

```
{  
  "amount": 100  
}
```

FRONTEND – REACT

Tạo project

```
npx create-react-app mediator-ui  
cd mediator-ui  
npm install axios
```

src/App.js

```
import React, { useState } from "react";  
import axios from "axios";  
  
function App() {  
  const [amount, setAmount] = useState(100);  
  const [result, setResult] = useState("");  
  
  const checkout = async () => {  
    const response = await axios.post(  
      "https://localhost:xxxx/api/orders/checkout",  
      {  
        amount: amount  
      }  
    );  
  
    setResult(response.data.message);  
  };  
  
  return (  
    <div style={{ padding: 40 }}>  
      <h2>Mediator Checkout</h2>  
  
      <label>Amount:</label>  
      <input  
        type="number"  
        value={amount}  
        onChange={ (e) => setAmount(e.target.value)}  
      />  
    </div>  
  );  
}
```



```
        <br /><br />

        <button onClick={checkout}>Checkout</button>

        <h3>{result}</h3>
      </div>
    );
  }
}

export default App;
```

▶ LUỒNG XỬ LÝ (MEDIATOR)

🔗 Khi user checkout:

1. React gửi request:

```
{
  "amount": 100
}
```

2. Controller gọi:

```
_mediator.Checkout(amount)
```

3. Mediator điều phối:

- CartService → Validate
- InventoryService → Reserve
- PaymentService → Pay
- NotificationService → Send

👉 TẤT CẢ đi qua Mediator

🔗 SO SÁNH NHANH (RẤT QUAN TRỌNG)

Trước (NO Mediator)	Sau (Mediator)
Service gọi nhau trực tiếp	Không service nào biết nhau
Coupling cao	Loose coupling
Khó maintain	Dễ mở rộng
Logic phân tán	Logic tập trung

MỞ RỘNG (BÀI TẬP CHO SINH VIÊN)

1. Thêm EmailService
2. Thêm DiscountService
3. Thêm LoggingService
4. Thêm nhiều loại checkout flow

 TẤT CẢ vẫn chỉ sửa Mediator